

Жизнь слишком коротка для медленного терминала

6 июня 2026 г.

Обновление: один из читателей прислал мне несколько отличных комментариев о том, как я здесь измеряю «скорость», а также по поводу некоторых моих рекомендаций. Я написал [продолжение](#) о том, как правильно проводить бенчмаркинг оболочек и в чем этот пост не дотягивает до идеала.

Практически вся моя работа выполняется в терминале. Git, kubectl, tmux, ssh на сервере — все это открыто практически весь день. То, чем я пользуюсь так часто, должно работать быстро. Любая задержка при открытии новой вкладки, вводе символа или нажатии клавиши Tab для автодополнения — это то, что я ощущаю сотни раз в день. Это смерть от тысячи порезов.

Моя оболочка запускается примерно за 30 миллисекунд:

```
$ for i in {1..5}; do /usr/bin/time zsh -i -c exit; done
0.03 real 0.02 user 0.01 sys
0.03 real 0.02 user 0.01 sys
...
```

Это полностью загруженная интерактивная оболочка с автодополнением, подсветкой синтаксиса, автопредложениями, fzf и direnv, которая запускается быстрее, чем один кадр при 30 кадрах в секунду. Новая вкладка открывается мгновенно. За этим не стоит какой-то масштабный проект по оптимизации. Я просто всегда старался, чтобы моя оболочка была минимальной и быстрой, и с годами это вошло в привычку. Вот как я это делаю, и все это можно найти в [моих dot-файлах](#).

Без фреймворка

Самое большое преимущество — это то, чего *нет*: ни oh-my-zsh, ни prezto, ни менеджера плагинов. Честно говоря, я никогда не понимал, чем так привлекательны эти фреймворки. Люди устанавливают oh-my-zsh с его сотнями плагинов и тем, в итоге используют, может

быть, 5% того, что он предлагает, а затем платят (своим временем и вычислительными ресурсами) за остальные 95% каждый раз, когда открывают командную строку. А менеджеры плагинов вдобавок к этому создают дополнительную нагрузку.

Я использую ровно три плагина, которые были клонированы из репозитория `git` с помощью моего скрипта установки и взяты из `.zshrc`:

```
source ~/.zsh/fzf-tab/fzf-tab.plugin.zsh
source ~/.zsh/zsh-autosuggestions/zsh-autosuggestions.zsh
source ~/.zsh/zsh-syntax-highlighting/zsh-syntax-highlighting.zsh
```

Здесь нет менеджера плагинов, который разрешал бы зависимости при запуске, а `source` файла, который уже находится на диске, практически ничего не стоит.

Кэширование автодополнения

`compinit` — одна из самых ресурсоёмких функций в типичном `.zshrc`. По умолчанию при каждом открытии оболочки выполняется проверка безопасности каждого файла автодополнения. Решение состоит в том, чтобы выполнять полный запуск только в том случае, если кэш (`.zcompdump`) старше 24 часов, а в остальных случаях пропускать проверку с помощью `-C`:

```
autoload -Uz compinit
if [[ -n ~/.zcompdump(#qNmh-24) ]]; then
    compinit -C
else
    compinit
fi
```

Этот квалификатор шаблона (`#qNmh-24`) означает «существует и был изменен в течение последних 24 часов». Таким образом, один полный `compinit` в день, а в остальное время используются данные из кэша.

Ленивая загрузка

`nvm` — пожалуй, самый известный «убийца» при запуске оболочки. Если загрузить его сразу, это может легко увеличить время запуска на полсекунды. Но `nvm` не нужен мне в каждой оболочке, он нужен мне, когда я ввожу `nvm`. Поэтому я оборачиваю его в функцию, которая заменяет себя при первом использовании:

```
export NVM_DIR="$HOME/.nvm"
nvm() {
  unset -f nvm
  [ -s "/opt/homebrew/opt/nvm/nvm.sh" ] && \. "/opt/homebrew/opt/nvm/nvm.sh" --no-use
  [ -s "/opt/homebrew/opt/nvm/etc/bash_completion.d/nvm" ] && \. "/opt/homebrew/c
  nvm "$@"
}
```

Первый вызов `nvm` удаляет заглушку, загружает настоящую версию (с помощью `--no-use`, чтобы не определять версию Node) и передает аргументы.

Та же идея применима к автодополнениям `kubectl`, которые запускают двоичный файл `kubectl` для создания скрипта автодополнения. Я загружаю их только после первого запуска `kubectl`:

```
kubectl() {
  command kubectl "$@"
  local ret=$?
  if [[ -z $KUBECTL_COMPLETE ]]; then
    source <(command kubectl completion zsh)
    KUBECTL_COMPLETE=1
  fi
  return $ret
}
```

This pattern works for a lot of things: anything that tells you to put `eval "$(tool init zsh)"` in your `.zshrc` is a candidate for lazy loading, because each of those forks a process and evaluates its output at startup. I keep `direnv` and `fzf` eager because they're fast and I use them constantly. Be strict about what you actually use a lot.

A non-blocking prompt

A prompt that runs `git status` synchronously will lag in any decently sized repo. That's the kind of lag you feel on every single Enter press, which is arguably worse than a slow startup. I use `pure`, which renders the prompt immediately and fills in the git info asynchronously when it's ready. I briefly tried replacing it with zsh's built-in `vcs_info`, but `pure`'s async behavior is just... better. You can do async git status in your own prompt as well, but `pure` wraps it rather nicely for my use-case.

The terminal itself

Shell startup is only half the story, because the emulator adds its own input latency. I use [Ghostty](#), which is GPU-accelerated and native, and my config is just seven lines long.

Combined with a `tmux new -A -s main alias (t)`, a fresh terminal window drops me right back into my existing session.

Measuring your own shell performance

You don't have to take my word for any of this, you can measure where the time goes in your own terminal. There are three kinds of lag to look for: startup time, prompt lag, and input latency.

Run this a few times (the first run is always slower because of cold caches):

```
time zsh -i -c exit
```

I think anything under 100ms is fine, and under 50ms is great. If you're seeing 500ms or more you have some work to do.

For proper statistics, use [hyperfine](#):

```
hyperfine --warmup 3 'zsh -i -c exit'
```

Zsh also ships with a profiler. Put this at the very top of your `.zshrc`:

```
zmodload zsh/zprof
```

and this at the very bottom:

```
zprof
```

Open a new shell and you get a sorted table of exactly where the time went. The top entries are usually `compinit`, an `nvm.sh` source, or some `eval "$(...)"`. Fix the top one, re-run, repeat. Remove both lines when you're done.

If `zprof` isn't granular enough, you can trace the whole startup with timestamps:

```
zsh -ixc exit 2>&1 | ts -i '%.s' | sort -rn | head -20
```

``set -x`` prints each trace line `_before_` running the command, so ``ts -i`` pins a command's duration on the line after it. The slow command is the one right above the big number in the original trace.

Or set `PS4='+%D{%s.%6.}: '` and run `zsh -ixc exit 2> startup.log`, then look for the big jumps between lines.

Startup can be fast while every prompt redraw is slow. `cd` into your biggest git repo and press Enter; if there's a delay before the next prompt appears, your prompt is doing synchronous work slowing it down. You can either switch to using an async prompt, or opt to strip out the Git functionality.

Wrapping up

Most of these optimizations are about leaving stuff out. It's about being intentional and only adding things you're going to use. Every one of the dozens of sessions I open per day is instant, and my terminal feels like an extension of my brain instead of an application I'm waiting on. For a tool I use the entire day that's non-negotiable for me.

All of the above lives in [my dotfiles repo](#) if you want to steal anything.

RELATED POSTS

I was wrong about fast terminals

A reader pushed back on my fast-terminal post, and they were right about more than I'd like. On benchmarking shells properly, modern zsh, and the difference between a shell that is fast and one that feels fast.

[Read post →](#)

Pro tips for remote working

Practical remote work tips from two years of working from home, covering focus time, breaks, workspace setup, and staying productive

[Read post →](#)

